

ZOOQUEST

A Jornada do Aprendizado

Relatório de Desenvolvimento - Versão Funcional do Jogo



Pré-visualização estrutural da tela inicial produzida a partir do layout do projeto.

Instituição	UniFil
Projeto	Práticas de Extensão - Jogos Digitais
Curso	Engenharia de Software
Tecnologia	Godot 4.x / GDScript / jogo 2D
Integrantes	Anaysa Nascimento, Daniel Yassuo, Davi Burke, Jehad Alshoura, Maria Eduarda Souza e Pedro Zaupa
Ano	2026

1. Resumo executivo

Esta entrega consolida o ZooQuest como uma versão funcional local do jogo no Godot. O repositório contém o fluxo completo entre menu, narrativa, transformações, três mini-games, retorno à forma humana, encerramento, pontuação e opção de jogar novamente.

ESTADO DECLARADO

VERSÃO FUNCIONAL LOCAL - TRÊS MINI-GAMES - FLUXO COMPLETO. A validação estática automatizada passou em todos os controles. A execução final deve ser confirmada no Godot da máquina de apresentação pelo roteiro incluído no repositório.

1.1 Objetivo educacional

O jogo utiliza matemática básica, raciocínio lógico, interpretação e responsabilidade para transformar conteúdo educativo em uma experiência lúdica para crianças. A narrativa ocorre durante uma excursão ao zoológico e relaciona escolhas, consequências e aprendizado.

1.2 Escopo desta versão

Componente	Estado	Resultado
Fluxo global	Implementado	Menu → narrativa → sapo → humano → pássaro → humano → minhoca → humano → final.
Sapo Matemático	Implementado	Seleção de vitória-régia e animação de salto para a alternativa.
Pássaro Lógico	Implementado	Escolha entre três passagens enquanto a barreira se aproxima.
Minhoca das Escolhas	Implementado	Movimento livre e coleta/click de frutas-resposta.
Perguntas e feedback	Implementado	Nove questões locais, explicação e repetição após erro.
Pontuação e replay	Implementado	10 pontos por acerto, máximo 90, estatísticas e reinício.

2. Requisitos da atividade e atendimento

A atividade de Desenvolvimento solicita uma versão funcional, as mecânicas essenciais e a navegação completa. A matriz abaixo mostra como cada exigência foi traduzida em artefatos verificáveis do repositório.

Exigência	Implementação no projeto	Como verificar
Versão funcional do jogo	<code>`project.godot`</code> inicia a jornada completa por <code>`core/main.tscn`</code> .	Importar o projeto e pressionar F5.
Mecânicas essenciais	As três mecânicas descritas no GDD estão separadas em <code>`minigames/`</code> .	Executar as três fases e testar acerto/erro.
Navegação completa	<code>`data/story_flow.json`</code> possui 13 etapas do início ao encerramento.	Concluir o fluxo sem abrir cenas manualmente.
Pontuação	<code>`GameState`</code> persiste pontos e estatísticas entre cenas.	Observar HUD e tela final.
Rejogabilidade	Tela final oferece jogar novamente e voltar ao menu.	Concluir uma partida e reiniciar.
Jogo 2D local	Recursos, perguntas e sons ficam no próprio repositório.	Validar ausência de API/backend e executar offline.
Documentação organizada	Fontes, diagramas, plano, relatório e testes estão em <code>`docs/`</code> .	Abrir <code>`CONTINUE_AQUI.md`</code> e seguir a ordem.

Decisão de escopo

A proposta antiga FindIt foi preservada apenas como histórico. Para a implementação, prevalecem a Visão ZooQuest, o GDD e a documentação de gameplay consolidada.

3. Fluxo funcional da jornada

Para o jogador, a experiência é contínua. Internamente, o Godot utiliza cenas especializadas, mas `Main` mantém o controle da sequência. Após cada mini-game, o personagem retorna à forma humana antes de receber a próxima transformação.



Figura 1 - Fluxo contínuo documentado para o ZooQuest.

3.1 Sequência executada pelo código

#	ID	Tipo	Função
1	intro	narrative	Uma excursao diferente
2	transform_frog	transformation	Transformacao: Sapo Matematico
3	frog_game	minigame	Mini-game frog
4	return_human_1	transformation	Primeiro desafio concluido!
5	between_frog_bird	narrative	A jornada continua
6	transform_bird	transformation	Transformacao: Passaro Logico
7	bird_game	minigame	Mini-game bird
8	return_human_2	transformation	Segundo desafio concluido!
9	between_bird_worm	narrative	A ultima licao
10	transform_worm	transformation	Transformacao: Minhoca das Escolhas
11	worm_game	minigame	Mini-game worm
12	return_human_3	transformation	A forma humana foi recuperada!
13	ending	ending	Encerramento e pontuação

4. Arquitetura do projeto Godot

A arquitetura separa o controle global das regras específicas de cada fase. Cada mini-game informa apenas que terminou; a decisão sobre a próxima etapa pertence a `core/main.gd`. Isso reduz acoplamento e permite evoluir uma fase sem reescrever as demais.

Princípio central

O mini-game conhece sua própria mecânica. O controlador principal conhece a ordem da jornada. O estado global conhece a pontuação. O conteúdo educativo permanece em arquivos de dados.

4.1 Componentes principais

Componente	Responsabilidade
core/main.gd	Carregar etapas, conectar sinais e avançar o fluxo.
core/game_state.gd	Manter pontuação, acertos, erros e fase atual.
data/question_bank.gd	Carregar e entregar perguntas por fase.
data/flow_repository.gd	Carregar a sequência narrativa local.
ui/hud.gd	Mostrar fase, pergunta, progresso, pontos e feedback.
minigames/*	Implementar a interação exclusiva de cada animal.

4.2 Estrutura resumida

```
ZooQuest_Projeto_Completo_Godot/
├── project.godot
├── core/
├── data/
├── ui/
├── characters/
├── minigames/
├── assets/
├── docs/
├── scripts/
└── tests/
```

5. Mini-game 1 - Sapo Matemático

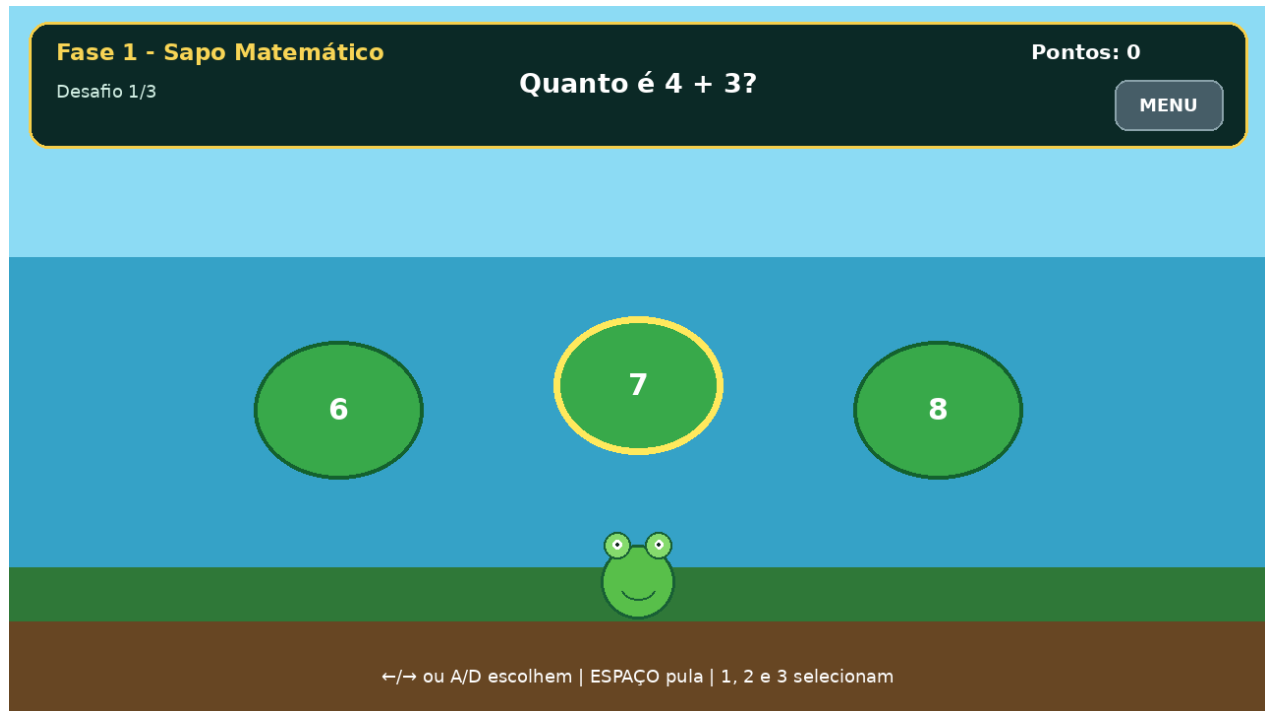


Figura 2 - Pré-visualização estrutural da fase do sapo; não é captura do runtime.

O jogador escolhe entre três vitórias-régias. As setas ou A/D alteram o destaque; ESPAÇO executa o salto. O clique e as teclas 1, 2 e 3 funcionam como alternativas de acessibilidade. A resposta incorreta faz o sapo afundar, mostra uma explicação e reinicia a mesma pergunta.

Elemento	Implementação
Controle	Esquerda/direita, A/D, ESPAÇO, clique ou 1-3.
Alternativas	Três instâncias de `lily_pad.gd`.
Feedback correto	Animação de pouso, som positivo, +10 pontos e próxima questão.
Feedback incorreto	Animação de afundamento, som de erro, explicação e repetição.
Conteúdo	Operações básicas de soma, subtração e multiplicação.

5.1 Arquivos-chave

`frog\_minigame.gd` coordena as perguntas; `frog\_player.gd` desenha e anima o sapo; `lily\_pad.gd` representa cada alternativa clicável.

6. Mini-game 2 - Pássaro Lógico



Figura 3 - Pré-visualização estrutural da fase do pássaro; não é captura do runtime.

O pássaro permanece à esquerda enquanto a barreira com três respostas se aproxima. O jogador muda de faixa com cima/baixo ou W/S. Ao atingir o ponto de validação, a faixa atual é comparada à alternativa correta. Essa solução representa progressão contínua sem depender de física complexa.

Elemento	Implementação
Controle	Cima/baixo, W/S ou seleção 1-3.
Progressão	A barreira desloca-se continuamente até o pássaro.
Feedback correto	Celebração, som positivo, +10 pontos e nova barreira.
Feedback incorreto	Efeito de colisão, explicação e reinício da tentativa.
Conteúdo	Sequências numéricas e identificação de padrões/formas.

6.1 Arquivos-chave

`bird\_minigame.gd` controla tempo e validação; `bird\_player.gd` controla faixas e animação; `answer\_gate.gd` representa as três passagens.

7. Mini-game 3 - Minhoca das Escolhas



Figura 4 - Pré-visualização estrutural da fase da minhoca; não é captura do runtime.

A minhoca utiliza movimento livre com `CharacterBody2D`. Cada fruta é uma `Area2D` que detecta a entrada do jogador e emite o índice da alternativa. O clique e as teclas 1-3 também registram a escolha. O conteúdo combina responsabilidade, interpretação e matemática simples.

Elemento	Implementação
Controle	Setas ou WASD; clique/1-3 como alternativas.
Alternativas	Três frutas com colisão, rótulo e cor distintos.
Feedback correto	Animação de comemoração, +10 pontos e próxima questão.
Feedback incorreto	Explicação da consequência e repetição.
Conteúdo	Cuidado com o ambiente, respeito às regras e interpretação.

7.1 Arquivos-chave

`worm_minigame.gd` coordena a fase; `worm_player.gd` realiza o movimento; `fruit_answer.gd` detecta colisão e clique.

8. Sistemas compartilhados e ciclo de gameplay

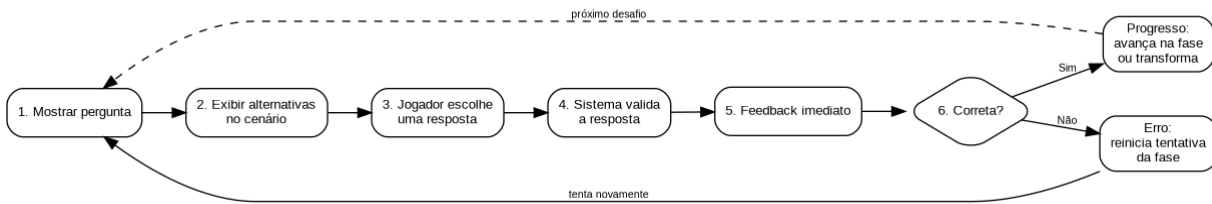


Figura 5 - Ciclo principal: pergunta, escolha, validação, feedback e consequência.

8.1 Banco de perguntas

As nove questões estão em `data/questions.json`. Cada registro possui ID, categoria, dificuldade, enunciado, três alternativas, índice correto e explicação. Separar dados e código permite revisar conteúdo sem alterar a lógica das fases.

8.2 Pontuação e feedback

Cada resposta correta vale 10 pontos. O erro não retira pontos: a tentativa é registrada, a explicação é exibida e a mesma questão reaparece. Essa decisão evita punição excessiva e preserva o aprendizado por tentativa e erro.

8.3 Navegação e retorno humano

O fluxo contém três etapas explícitas de retorno à forma humana. Isso implementa a correção definida pelo grupo: cada animal representa um desafio completo e, após vencê-lo, o personagem recupera temporariamente sua forma antes da transformação seguinte.

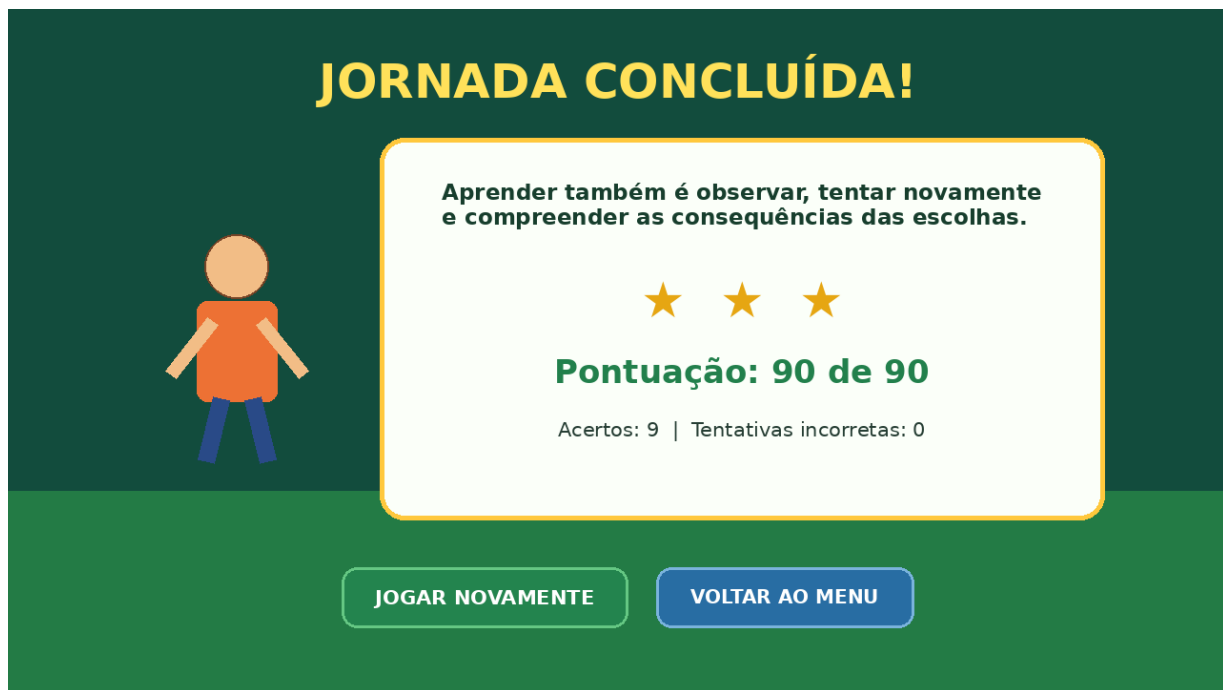
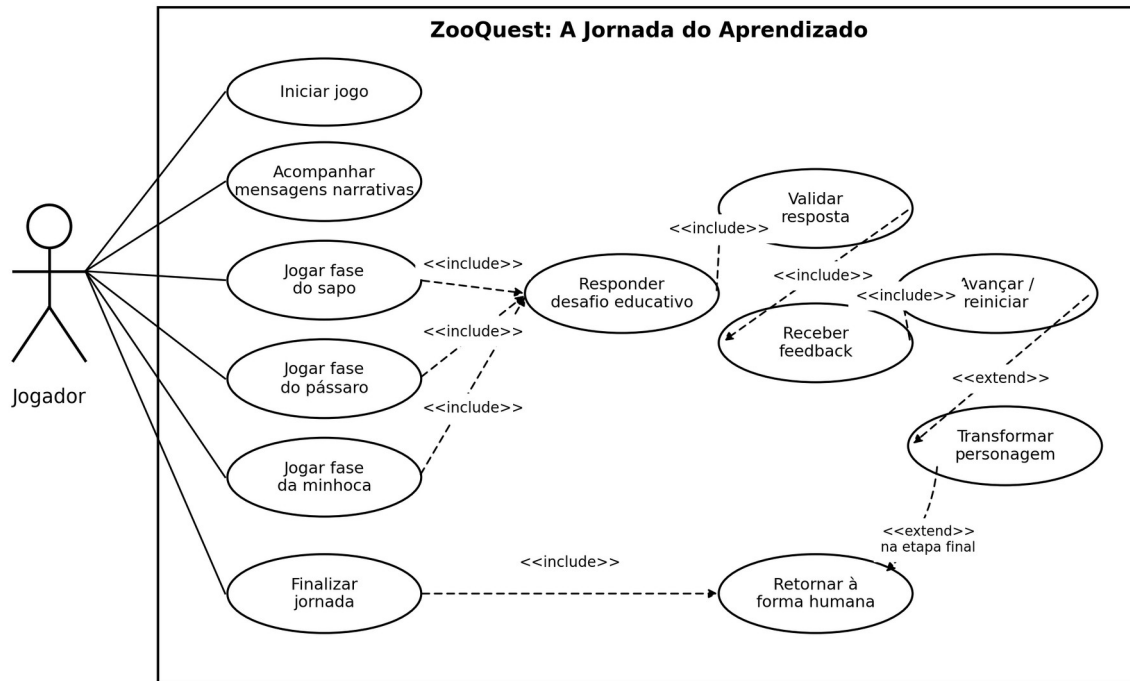


Figura 6 - Pré-visualização estrutural do encerramento e da rejogabilidade.

9. Caso de uso e interação jogador-sistema

Diagrama de Caso de Uso - ZooQuest



Observação: validação, feedback, transformação e reinício são comportamentos do sistema durante as fases ou entre elas.

Figura 7 - Diagrama de caso de uso consolidado do ZooQuest.

O ator principal inicia a jornada, acompanha mensagens, joga as três fases e recebe feedback. A validação, o avanço, o reinício e as transformações são comportamentos internos do sistema realizados durante ou entre as fases.

10. Validação e qualidade

Foi executado o validador local `scripts/validate\_repository.py`, que verifica estrutura, arquivos obrigatórios, referências `res://`, contratos JSON, quantidade e unicidade de perguntas, ordem do fluxo, retorno humano, integridade básica dos scripts, cenas, sons e fontes documentais.

RESULTADO DA VALIDAÇÃO ESTÁTICA

PASS - 91 de 91 controles aprovados; 0 erros e 0 avisos.

10.1 Limite da validação realizada

O ambiente usado para gerar o pacote não possuía o executável do Godot. Portanto, não se afirma que houve execução gráfica do jogo nesse ambiente. O repositório inclui `scripts/run\_godot\_smoke.sh`, `tests/smoke\_test.gd` e um roteiro manual com 30 casos para completar a validação no computador do grupo.

10.2 Gates para a apresentação

Gate	Critério
G1 - Importação	Godot importa o projeto sem erro de parse.
G2 - Fluxo	F5 percorre todas as etapas sem abrir cenas manualmente.
G3 - Mecânicas	Acerto e erro funcionam nas três fases.
G4 - Estado	Pontuação e estatísticas permanecem entre as fases.
G5 - Rejogabilidade	Jogar novamente e voltar ao menu funcionam.
G6 - Debugger	Nenhum erro vermelho durante uma partida completa.

11. Organização do repositório e trabalho em equipe

O repositório incorpora o padrão de retomada e rastreabilidade: `README.md`, `CONTINUE\_AQUI.md`, estado atual, arquitetura, plano mestre, fontes, diagramas, relatório, testes e validação. Os documentos recebidos foram copiados para `docs/fontes/`, preservando o histórico e a justificativa das decisões.

11.1 Distribuição operacional

Frente	Carga	Responsabilidade
Lead técnico e integração	70%	Arquitetura, fluxo, estado, perguntas, integração, correções e fechamento.
Sapo	7,5%	Revisão visual/funcional da primeira fase.
Pássaro	7,5%	Revisão visual/funcional da segunda fase.
Minhoca	7,5%	Revisão visual/funcional da terceira fase.
Conteúdo e QA	7,5%	Perguntas, textos, testes, capturas e relatório.

11.2 Materiais preservados

Arquivo	Finalidade
Orientacoes_UniFil_Jogos.pdf	Requisitos institucionais.
Visao_ZooQuest.pdf	Visão detalhada do produto.
GDD_ZooQuest.pdf	Narrativa, fases, regras e pontuação.
Documentacao_Gameplay_e_Fluxos.docx	Caso de uso, fluxo e interação.
Casos_de_Uso_ZooQuest.asta	Modelo editável do Astah.
Projeto_PI_FindIt_Historico.pdf	Proposta inicial preservada como histórico.

12. Procedimento imediato de execução

Passo	Ação
1	Extrair o ZIP sem renomear ou mover arquivos internos.
2	Abrir o Godot 4.x e importar o `project.godot` da raiz.
3	Aguardar a importação dos WAVs e do ícone.
4	Pressionar F5 e executar uma partida completa.
5	Seguir `tests/manual/ROTEIRO_TESTE_FUNCIONAL.md`.
6	Registrar qualquer erro do Debugger antes de alterar o código.
7	Executar `python3 scripts/validate_repository.py` após alterações.

13. Conclusão

A versão entregue traduz as documentações do ZooQuest em um projeto Godot organizado e jogável, cobrindo as três mecânicas essenciais e a navegação integral da jornada. O código foi planejado para permitir correção rápida, estudo por arquivo e evolução posterior de arte, áudio e conteúdo sem perder a separação de responsabilidades.

A validação automatizada confirma a integridade estrutural do pacote. A próxima ação não é replanejar: é abrir o projeto no Godot, executar os 30 casos do roteiro e corrigir somente eventuais divergências observadas no runtime da máquina do grupo.

Referências documentais

- UNIFIL. Orientações para a criação de jogos digitais. 2026.
- ZOOQUEST: A Jornada do Aprendizado. Documento de Visão. 15 maio 2026.
- ZOOQUEST: A Jornada do Aprendizado. Game Design Document. 2026.
- ZOOQUEST. Documentação de Caso de Uso, Fluxo de Jogo, Interação do Jogador e Ciclo Principal do Gameplay. 2026.